# PAMS: Testing Document

## Group 28

**Module:** UFCF8S-30-2 Advanced Software Development

**Year:** 2025/26

**Assessment:** Portfolio (Element 3, Testing)

**Submission Date:** 23 April 2026

### Group Members


| Name | Student ID |
|---|---|
| Omar Al Frouh | 24020879 |
| Feras Bader | 24033513 |
| Ahmed Deraz | 24046227 |
| Numair | 24026758 |

1. Test Strategy....3
2. Manual Test Cases....3
2.1 Authentication (AUTH)....3
2.2 User Management (USR)....6
2.3 Apartment Management (APT)....8
2.4 Tenant Management (TEN)....10
2.5 Invoice Management (INV)....14
2.6 Payment Management (PAY)....16
2.7 Maintenance Management (MNT)....20
2.8 Reports (RPT)....22
2.9 Notifications (NOTIF)....24
2.10 Security (SEC)....25
2.11 Integration Flows (INT)....26
3. Automated Testing....27
3.1 Framework....27
3.2 Coverage Summary....27
3.3 Run Evidence....28
4. Conclusion....29

# 1. Test Strategy

We tested PAMS with two strategies running together. Manual testing drives the graphical user interface one screen at a time, with a screenshot captured for each case as evidence. Automated testing uses Python's standard `unittest` framework to lock down business rules, input validation and edge cases at the manager layer. The two strategies cover different risks: the manual suite proves the system works as an application, and the automated suite proves the logic underneath stays correct when the code changes.

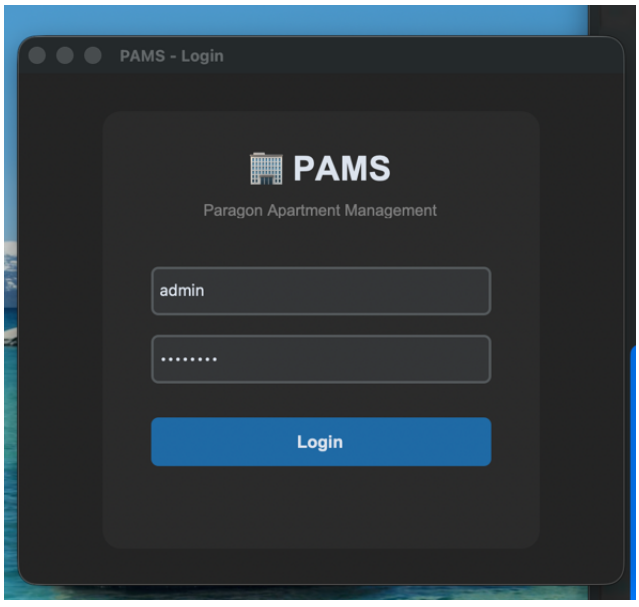
The manual cases in Section 2 cover every component listed in the case study (account and user management, tenant, apartment, invoice and payment, maintenance, reporting) plus cross-cutting concerns (security and end-to-end integration). Cases include happy paths, input validation (empty fields, negative numbers, wrong types), business rule enforcement (5% early termination penalty, 30 day notice, no double payment, no invoicing of terminated tenants) and deliberate attempts to break the system with bad data.

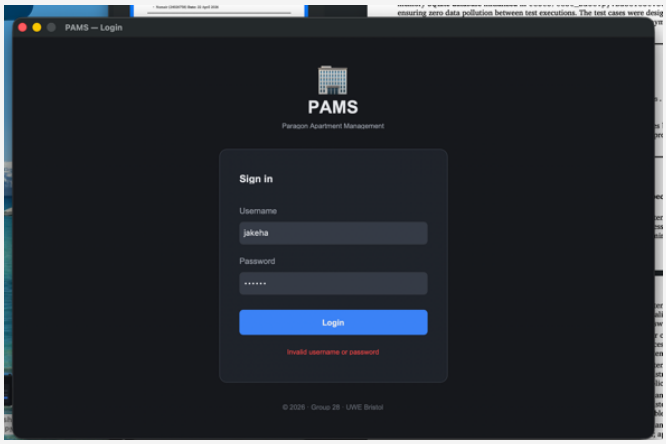
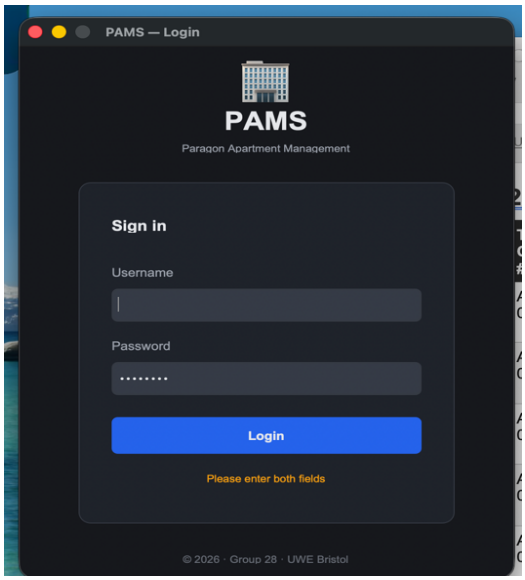
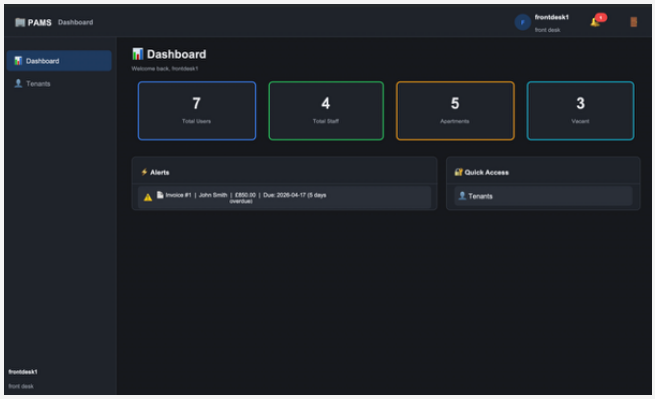
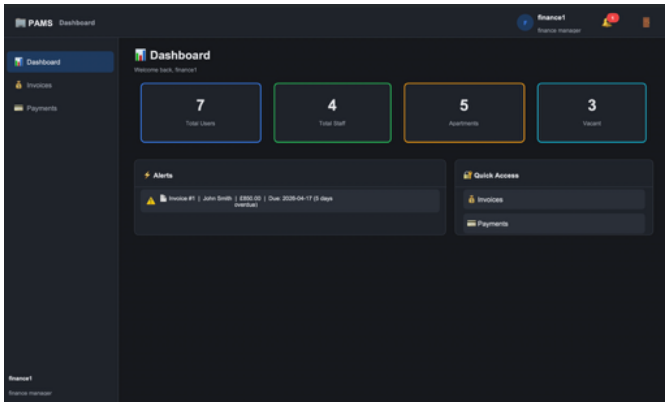
Automated tests extend `tests/test_base.py:BaseTest`, which creates a fresh SQLite file at `tests/test_temp.db` in `setUp()` and removes it in `tearDown()`. Every test therefore runs against a clean schema, in any order, with no state carried from one test to the next.

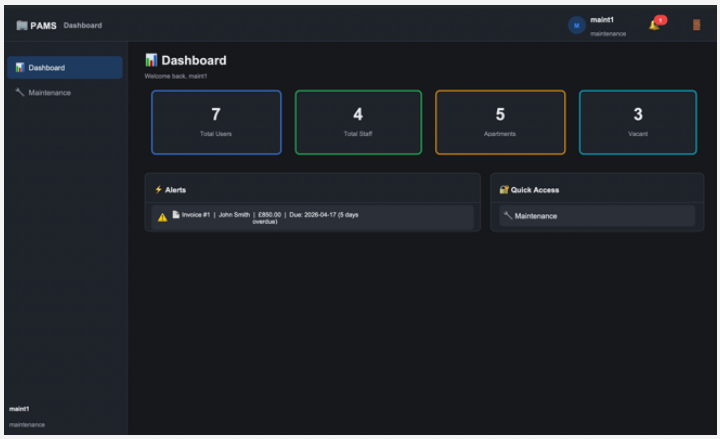
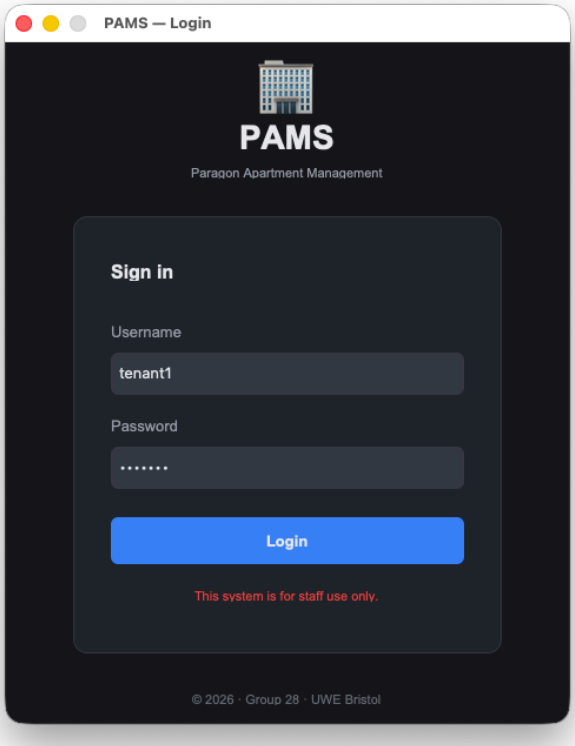
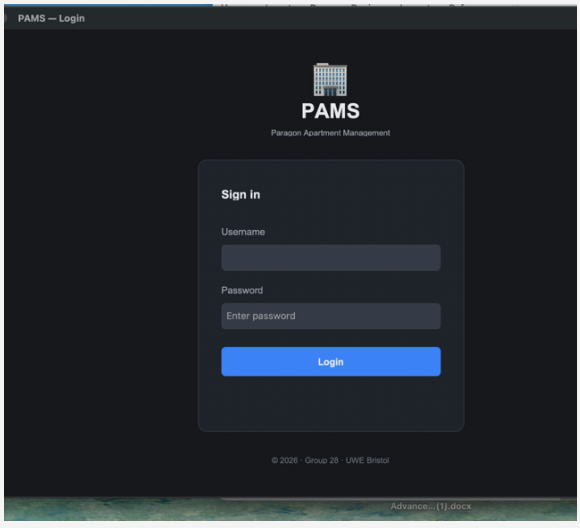
**Screenshot workflow.** Each screenshot below was taken on a freshly seeded database (`python3 seed_db.py` after removing any existing `pams.db`), with the app running via `python3 main.py`, the action in the Test Dataset/Input column performed exactly as written, and the screen captured at the moment the system responded.

# 2. Manual Test Cases

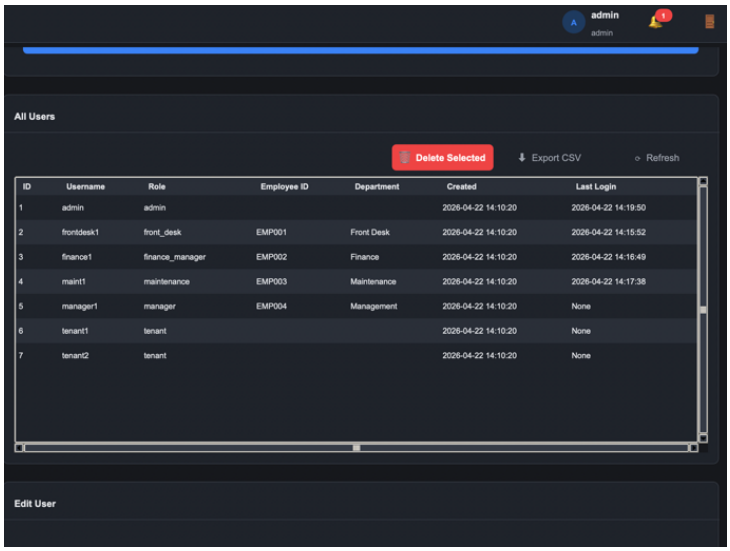
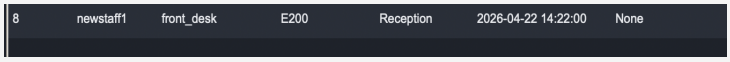
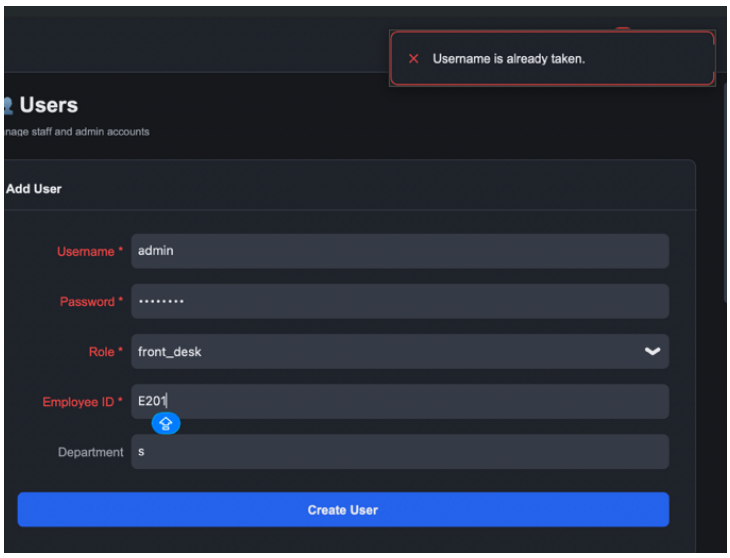
## 2.1 Authentication (AUTH)

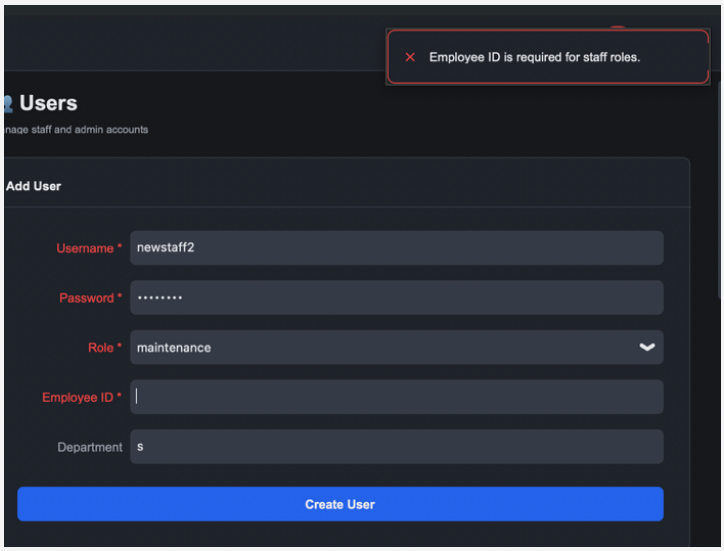
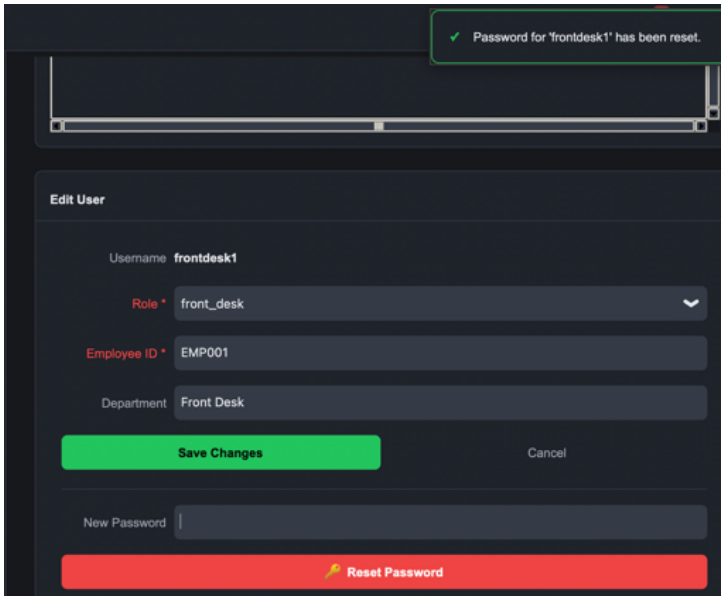
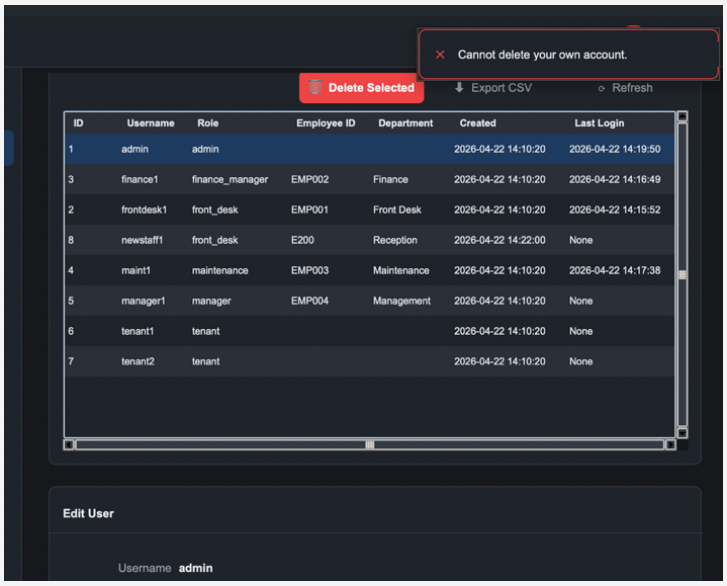
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| AUTH-01 | Log in with valid admin credentials | Username: `admin`, Password: `admin123` |  | Login succeeds. Main window opens with the Admin sidebar. |

| | | | | |
|---|---|---|---|---|
| AUTH-02 | Log in with invalid password | Username: `admin`, Password: `wrongpass` |  | Login fails. Status label shows "Invalid username or password." |
| AUTH-03 | Log in with empty username | Username: (empty), Password: `admin123` |  | Login fails with a validation message. |
| AUTH-04 | Log in as front desk staff | Username: `frontdesk1`, Password: `pass123` |  | Login succeeds. Sidebar shows only Dashboard and Tenants. |
| AUTH-05 | Log in as finance manager | Username: `finance1`, Password: `pass123` |  | Login succeeds. Sidebar shows Dashboard, Invoices, Payments only. |

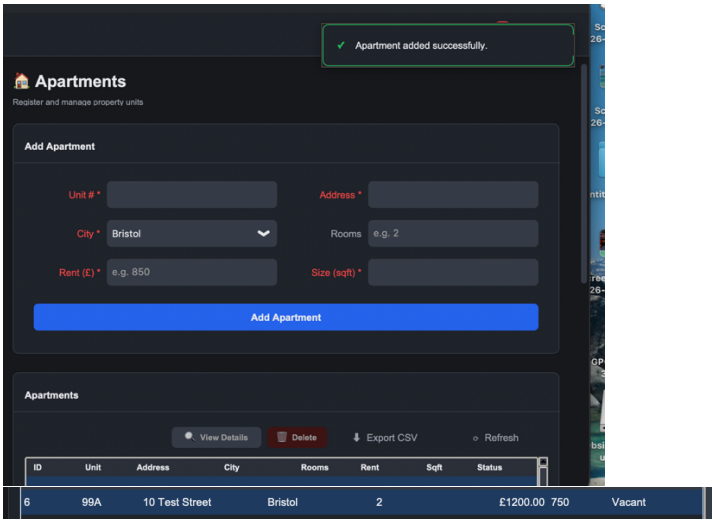
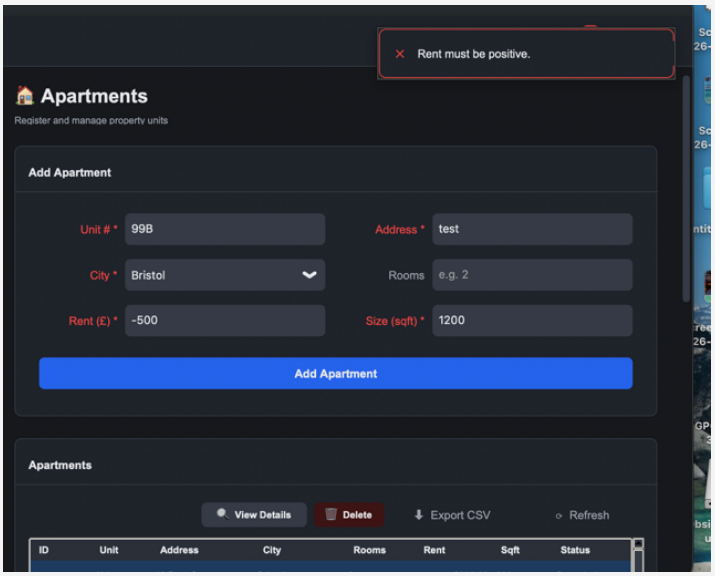
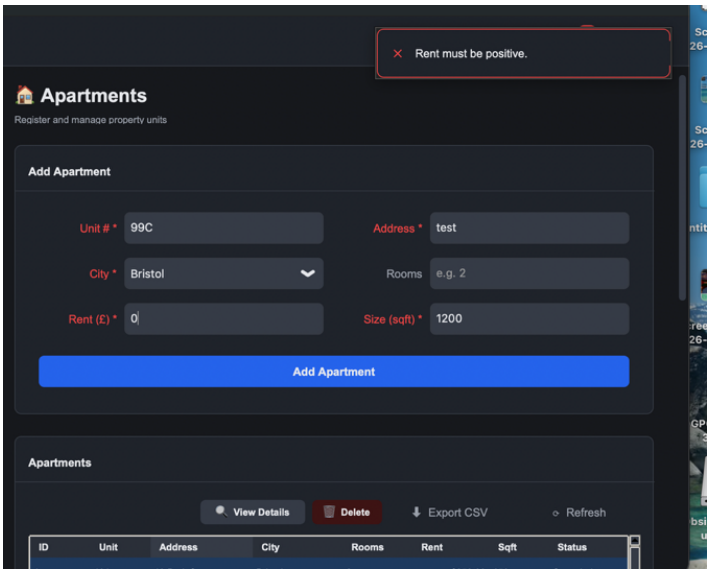
| | | | | |
|---|---|---|---|---|
| AUTH-06 | Log in as maintenance staff | Username: `maint1`, Password: `pass123` |  | Login succeeds. Sidebar shows Dashboard and Maintenance only. |
| AUTH-07 | Tenant role is blocked at login | Valid tenant record credentials |  | Login is rejected with the message "This system is for staff use only." |
| AUTH-08 | Logout from header | Click the logout icon in the top right header |  | Confirmation dialog appears. Confirming closes the main window and reopens the login screen. |

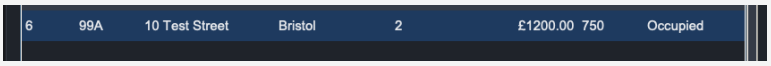
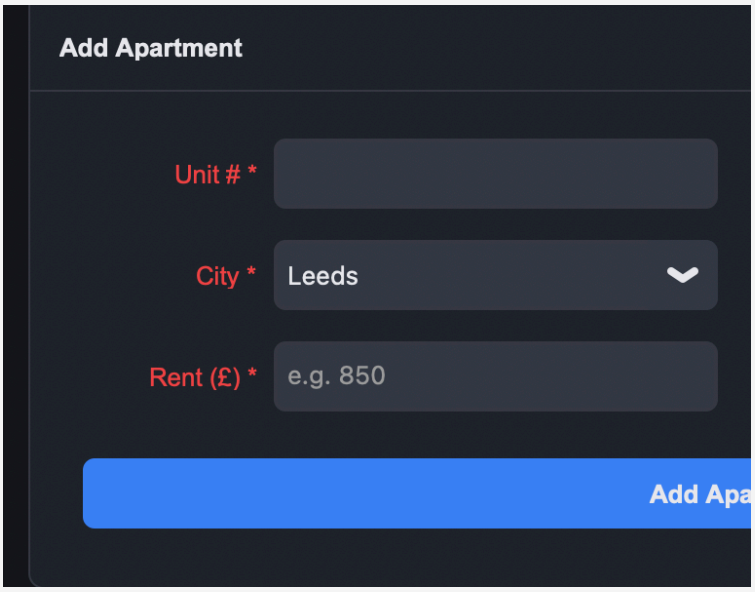
## 2.2 User Management (USR)

| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| USR-01 | View the list of all users | Navigate to Users tab as admin |  | Table lists every user with ID, Username, Role, Employee ID, Department, Created, Last Login columns. |
| USR-02 | Add a new staff user | Username: `newstaff1`, Password: `pass123`, Role: `front_desk`, Employee ID: `E200`, Department: `Reception` |  | Success toast. New user appears in the table with the correct role. |
| USR-03 | Reject duplicate username | Username: `admin` (already exists), Role: `front_desk`, Employee ID: `E201` |  | Error toast: "Username is already taken." No row is added. |

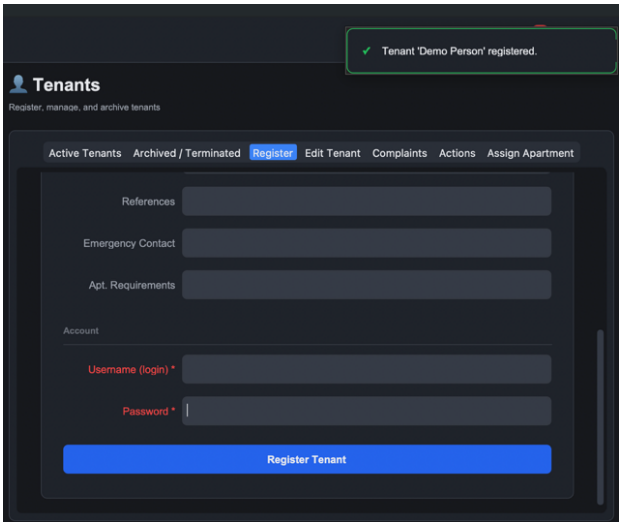
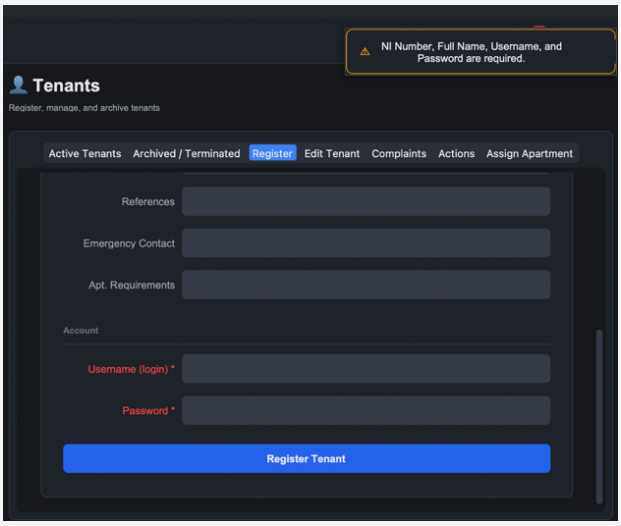
| | | | | |
|---|---|---|---|---|
| USR-04 | Reject empty Employee ID for staff role | Username: `newstaff2`, Role: `maintenance`, Employee ID: (empty) |  | Error toast: "Employee ID is required for staff roles." |
| USR-05 | Reset another user's password | Select user, click Reset Password, enter `newpass456` |  | Success toast. Logging in with the new password succeeds. |
| USR-06 | Prevent deleting the only admin | Select the sole admin account, click Delete |  | Error toast: "Cannot remove the only admin account." |

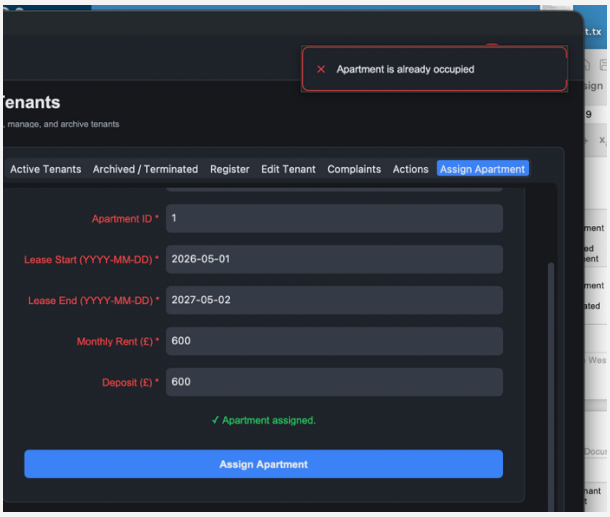
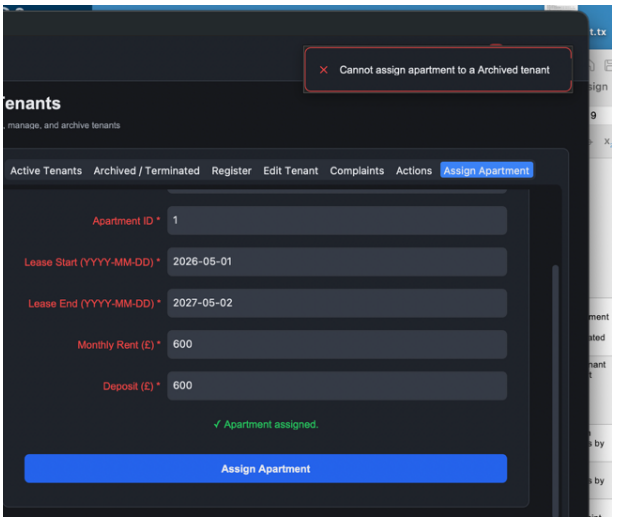
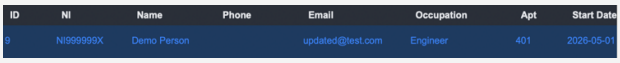
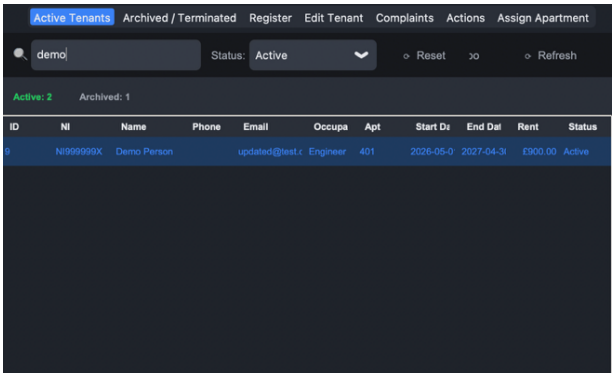
## 2.3 Apartment Management (APT)

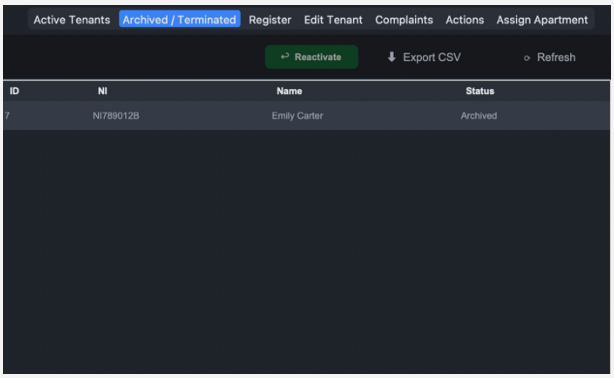
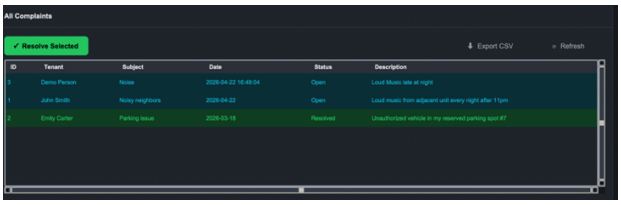
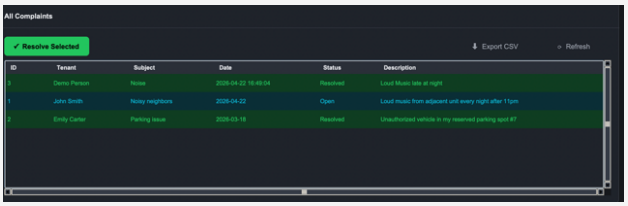
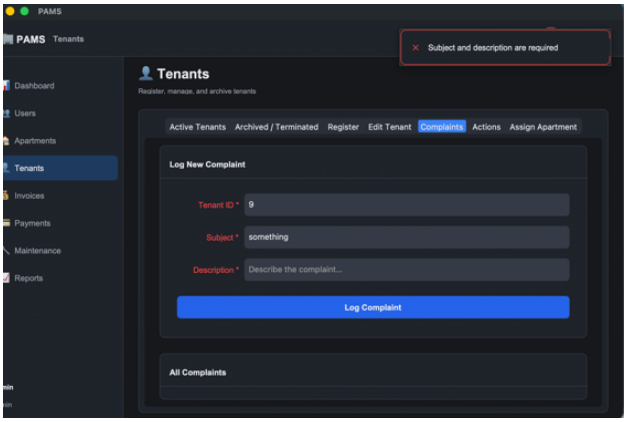
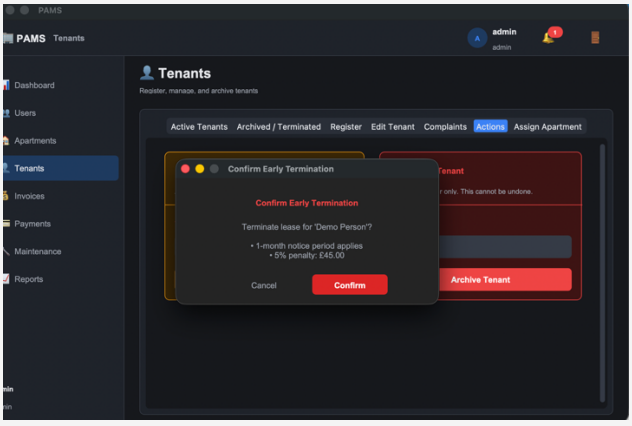
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| APT-01 | Register a new apartment | Unit: `99A`, Address: `10 Test Street`, City: `Bristol`, Rooms: `2`, Rent: `1200`, Sqft: `750` |  | Success toast. New row appears with status `Vacant`. |
| APT-02 | Reject a negative rental price | Unit: `99B`, Rent: `-500`, other fields valid |  | Error toast: "Rental price must be positive." No row is added. |
| APT-03 | Reject zero rental price | Unit: `99C`, Rent: `0`, other fields valid |  | Error toast: "Rental price must be positive." |

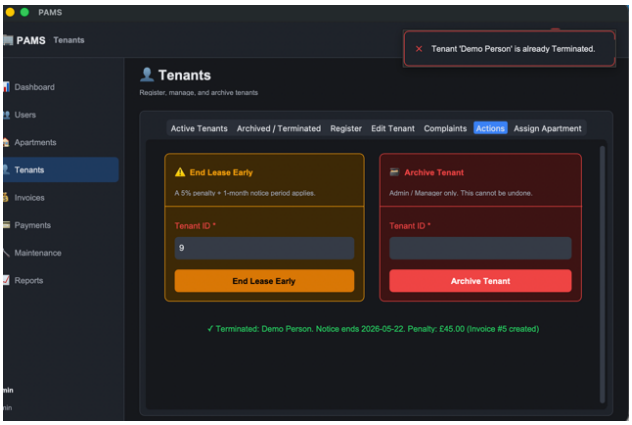
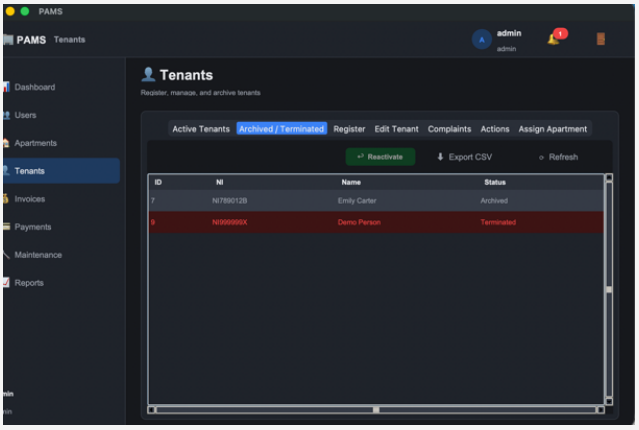
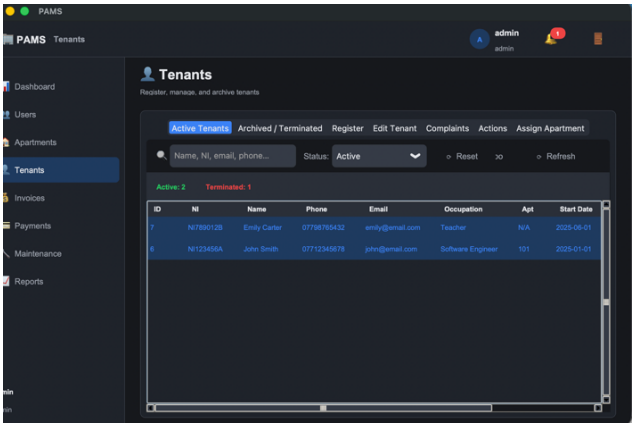
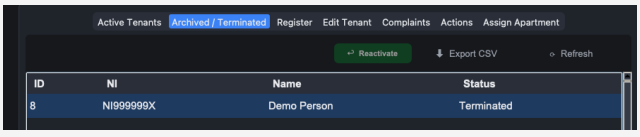
| | | | | |
|---|---|---|---|---|
| APT-04 | Status changes to Occupied via tenant assignment | Go to Tenants → Assign Apartment tab → assign a tenant to a Vacant apartment → go to Apartments → verify status now shows "Occupied" |  | Status column shows "Occupied", apartment no longer appears in available assignment list |
| APT-05 | Status reverts to Vacant after tenant archived | Go to Tenants → Actions tab → Archive a tenant who has an apartment → go to Apartments → verify that apartment status is back to "Vacant" |  | Status column shows "Vacant", apartment is now available for assignment |
| APT-06 | City dropdown picks up new cities | Register an apartment in a new city `Leeds`, reopen the form |  | `Leeds` appears as an option in the city dropdown. |

## 2.4 Tenant Management (TEN)

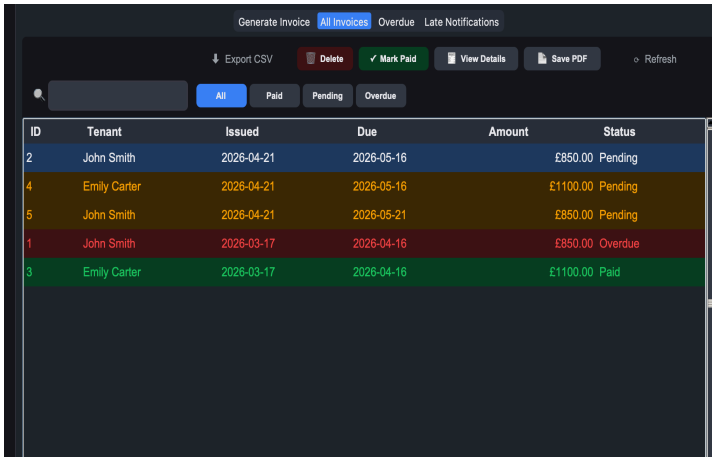
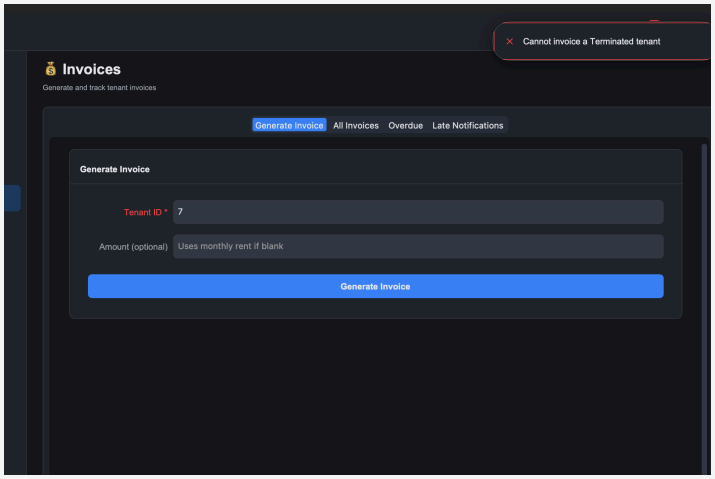
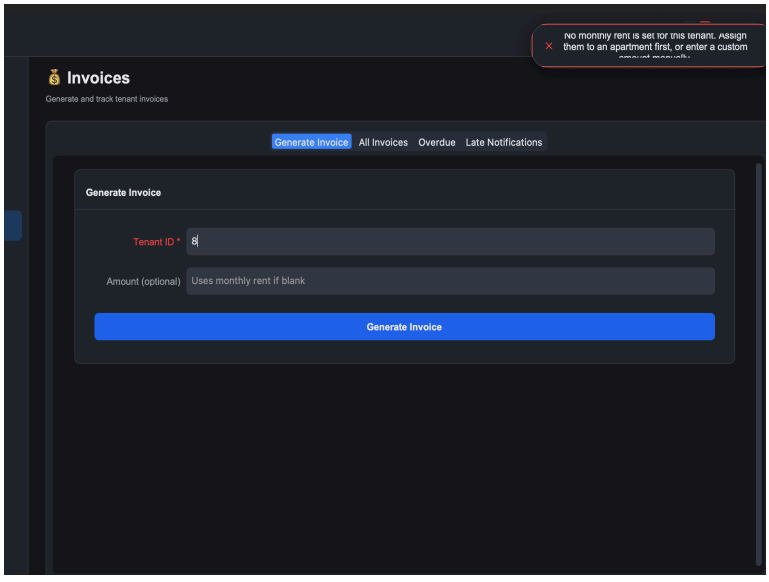
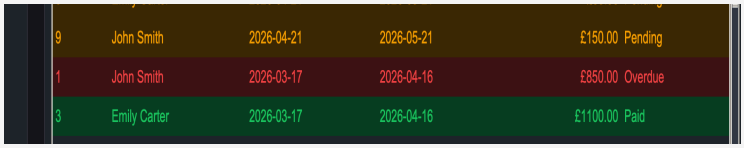
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| TEN-01 | Register a new tenant | NI: `NI999999X`, Name: `Demo Person`, Phone: `07000000001`, Email: `demo@test.com`, Occupation: `Engineer`, References: `Previous landlord`, Emergency: `John 07000000002`, Requirements: `2 bed` |  | Success toast. Tenant appears in Active Tenants with status `Active`. |
| TEN-02 | Reject registration with empty NI | Name: `Demo Person`, NI: (empty), other fields valid |  | Validation error. No tenant is created. |
| TEN-03 | Assign an apartment to a tenant | Tenant: `Demo Person`, Apartment: `102`, Lease: `2026-05-01` to `2027-04-30`, Rent: `900`, Deposit: `900` |  | Tenant row shows Unit `102` and rent £900.00. Apartment `102` flips from `Vacant` to `Occupied`. |

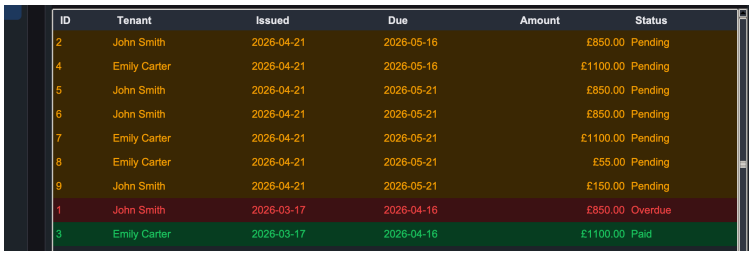
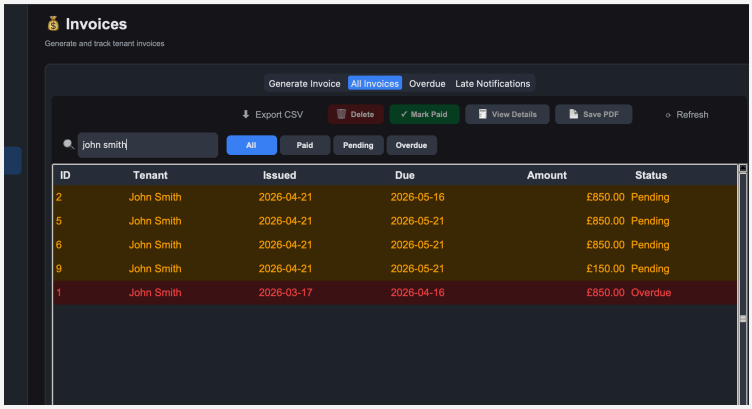
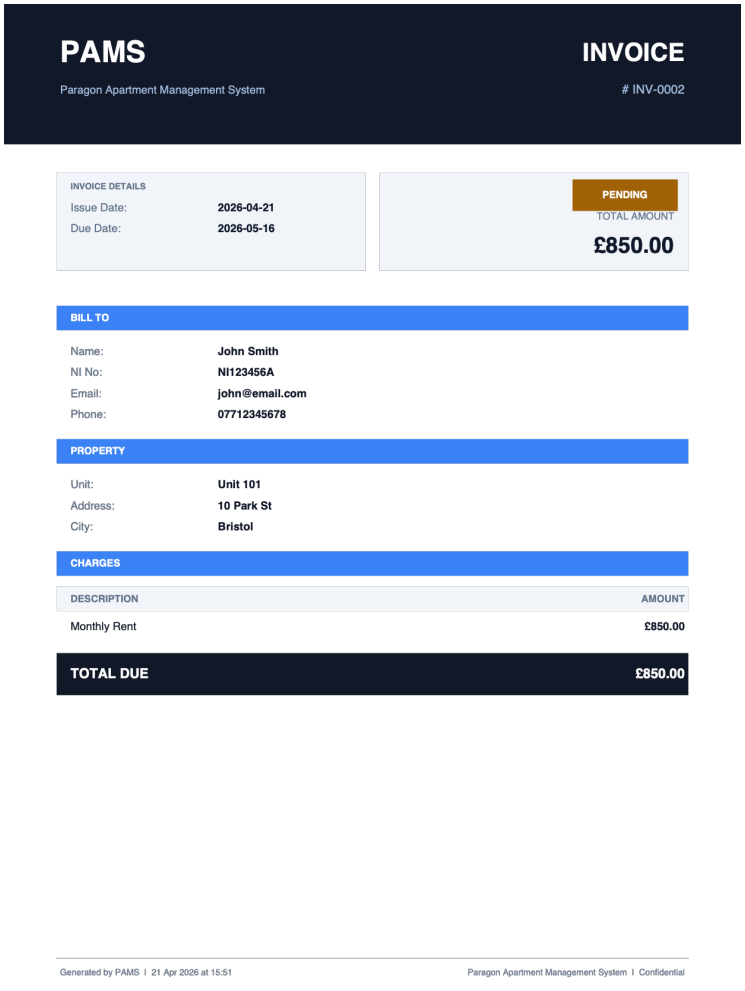
| | | | | |
|---|---|---|---|---|
| TEN-04 | Block assignment to an occupied apartment | Tenant: `Demo Person 2`, Apartment: `101` (already occupied) |  | Error toast: "Apartment is already occupied." |
| TEN-05 | Block assignment to a terminated tenant | Select a terminated tenant, try Assign |  | Error toast: "Cannot assign apartment to a Terminated tenant." |
| TEN-06 | Edit tenant contact details | Load tenant, change email to `updated@test.com`, Save |  | Success message. Active Tenants table shows the new email on refresh. |
| TEN-07 | Search tenants by name | Type `Demo` in the search bar |  | Table filters to matching rows. |

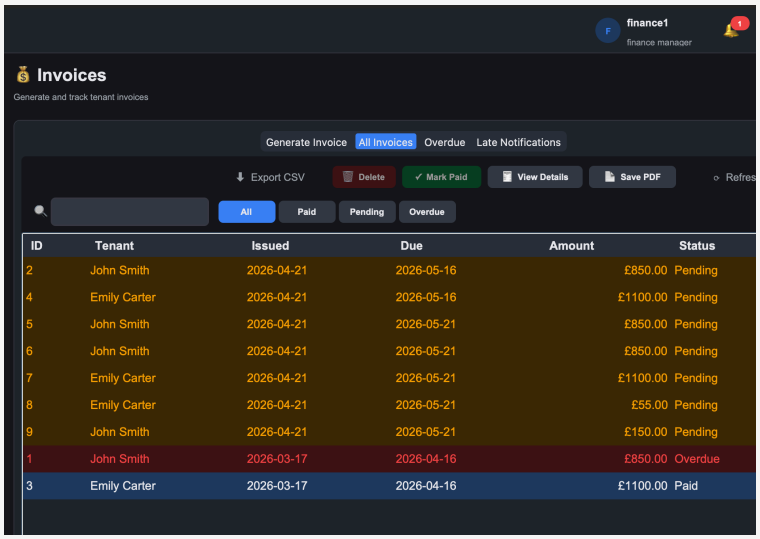
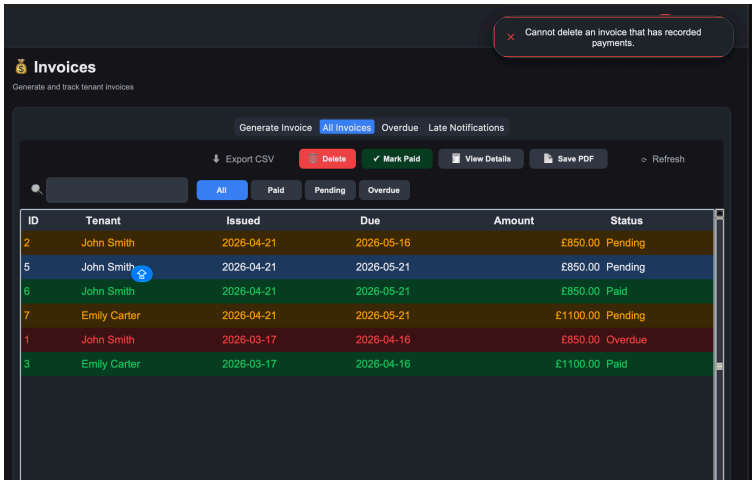
| | | | | |
|---|---|---|---|---|
| TEN-08 | Filter tenants by status | Select `Archived` in the status control |  | Table switches to archived tenants only. |
| TEN-09 | Log a complaint | Tenant: `Demo Person`, Subject: `Noise`, Description: `Loud music late at night` |  | Success toast. Complaint appears in the Complaints tab with status `Open`. |
| TEN-10 | Resolve a complaint | Select open complaint, click Resolve Selected |  | Status flips to `Resolved` and `resolved_date` is set to today. |
| TEN-11 | Reject empty complaint subject | Subject: (empty), Description: `something` |  | Error toast: "Subject and description are required." |
| TEN-12 | Early termination applies 5% penalty and 30 day notice | Tenant: `Demo Person` (rent £900), confirm Early Termination |  | Popup shows penalty £45.00 and effective date of today plus 30 days. Tenant status flips to `Terminated`. Apartment flips to `Vacant`. |

| | | | | |
|---|---|---|---|---|
| TEN-13 | Block early termination of a terminated tenant | Select terminated tenant again, Early Terminate |  | Error toast: "Tenant is already Terminated." |
| TEN-14 | Archive a tenant | Select active tenant, Archive, confirm |  | Tenant moves to Archived tab. Apartment becomes `Vacant`. |
| TEN-15 | Reactivate an archived tenant | Select archived tenant, Reactivate |  | Tenant returns to Active Tenants with status `Active`. |
| TEN-16 | Block reactivation of a terminated tenant | Select terminated tenant, Reactivate |  | Reactivate button greyed out |

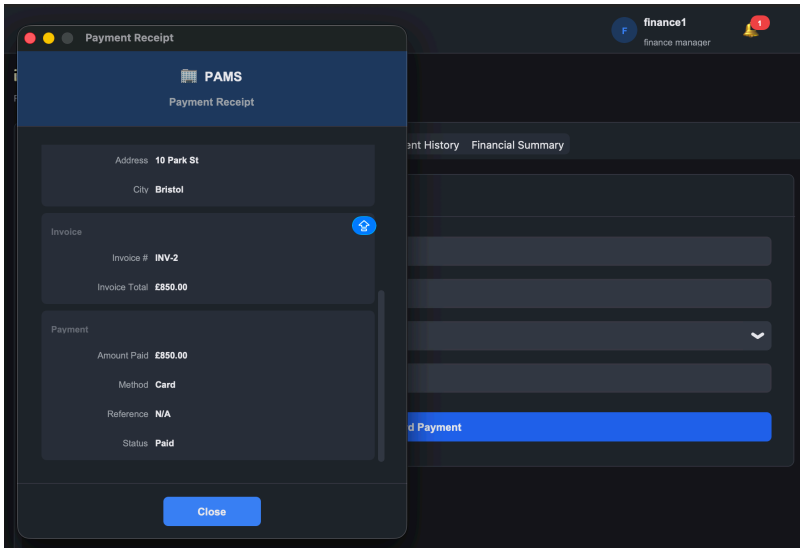
## 2.5 Invoice Management (INV)

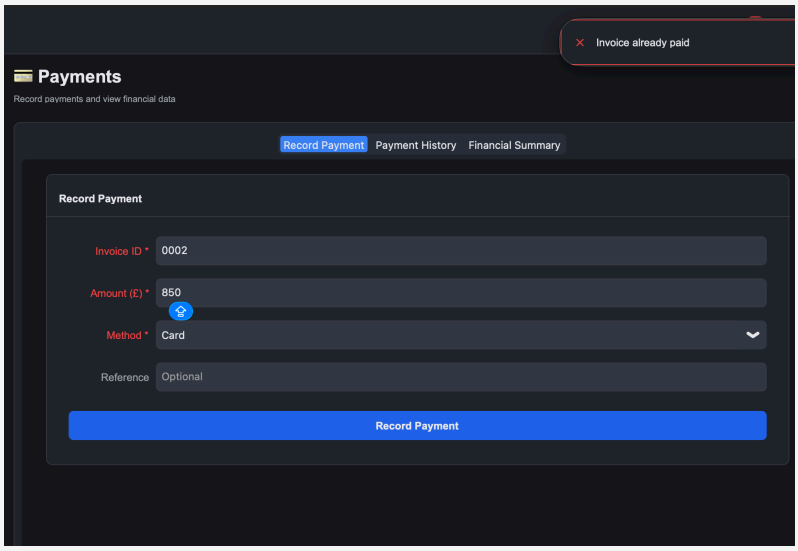
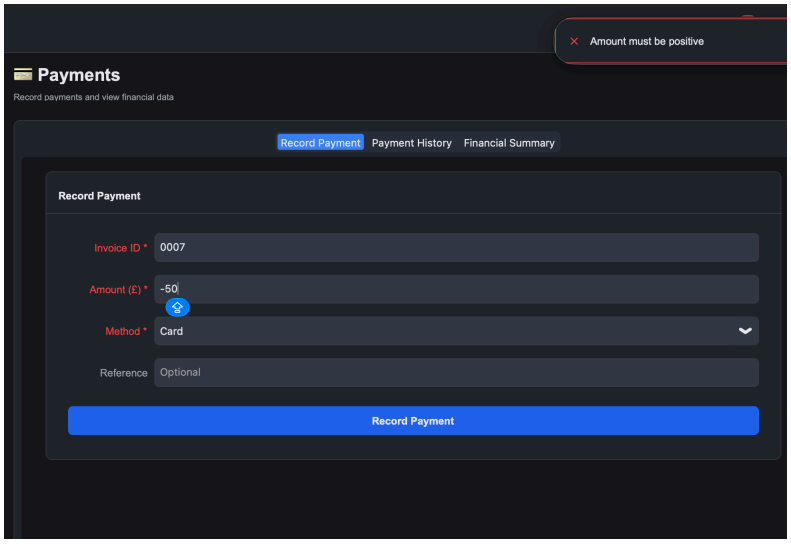
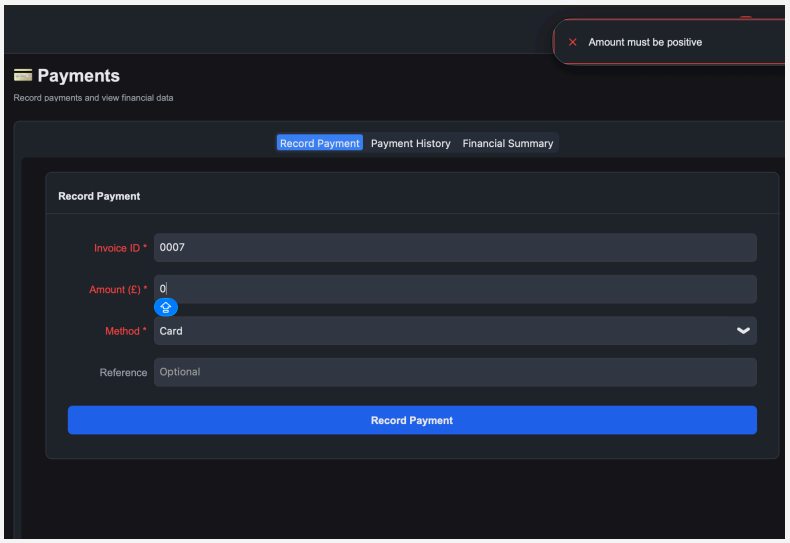
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| INV-01 | Generate monthly rent invoice | Active tenant `John Smith` with rent £850, click Generate Invoice |  | Success toast with new invoice ID. Invoice appears with status `Pending` and due date of today plus 30 days. |
| INV-02 | Block invoice for terminated tenant | Select terminated tenant, Generate Invoice |  | Error toast: "Cannot invoice a Terminated tenant." |
| INV-03 | Block invoice when tenant has no monthly rent | Tenant with `monthly_rent` NULL, custom amount blank |  | Error toast explains the tenant has no rent set and offers a fix. |
| INV-04 | Generate custom amount invoice | Tenant: `John Smith`, Custom amount: `150.00` |  | Invoice is created with total £150.00 instead of |

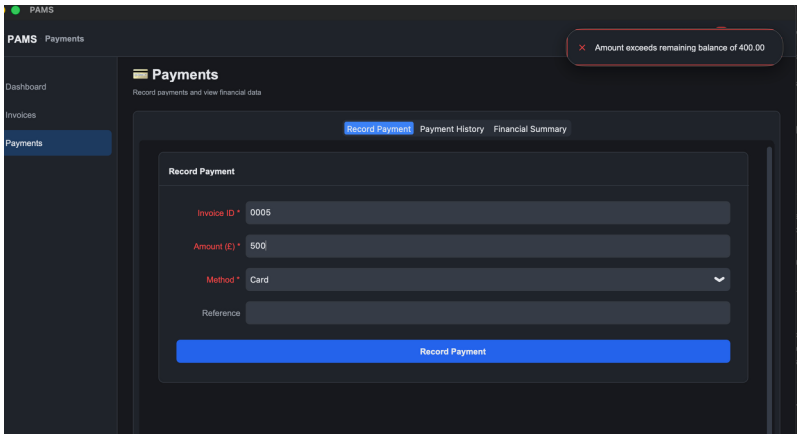
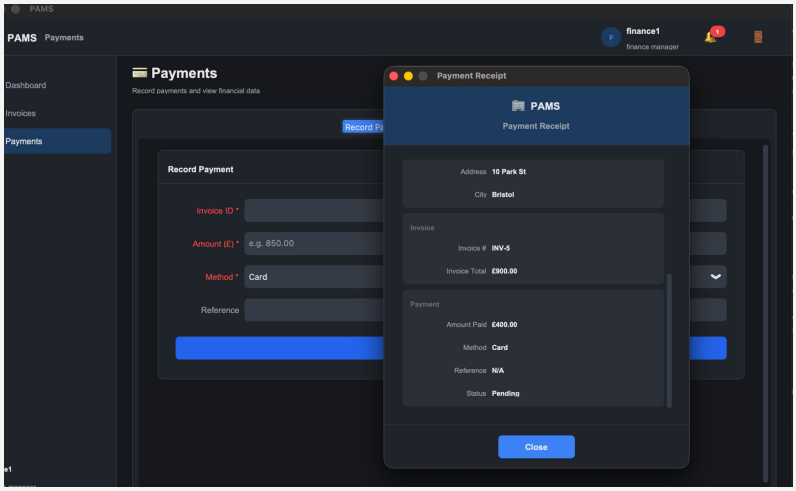
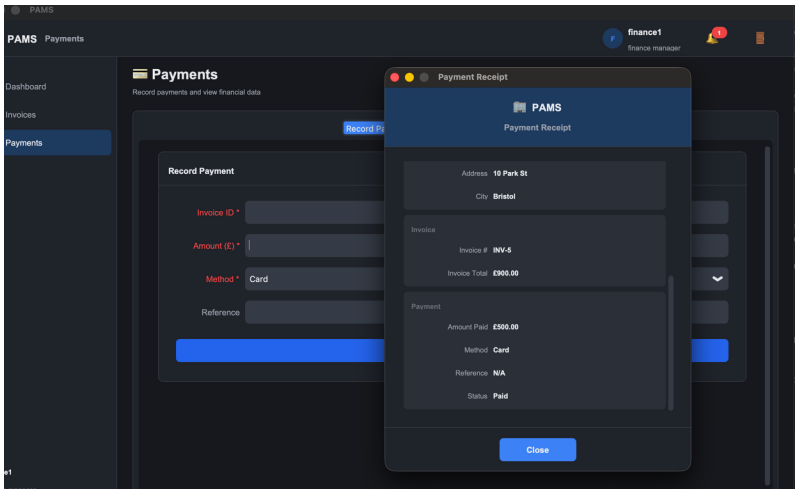
| | | | | |
|---|---|---|---|---|
| | | | | the monthly rent. |
| INV-05 | Overdue badge appears on past due invoices | Seed state includes an invoice with due date in the past |  | Status column renders an `Overdue` red pill. Overdue tab lists the same invoice. |
| INV-06 | Search invoices by tenant name | Type `John` in the invoices search bar |  | Table filters to invoices whose tenant name contains `John`. |
| INV-07 | Export invoice as PDF | Select invoice, Export PDF, choose a save path |  | PDF is written and opens cleanly showing tenant info, dates, total and payments. |

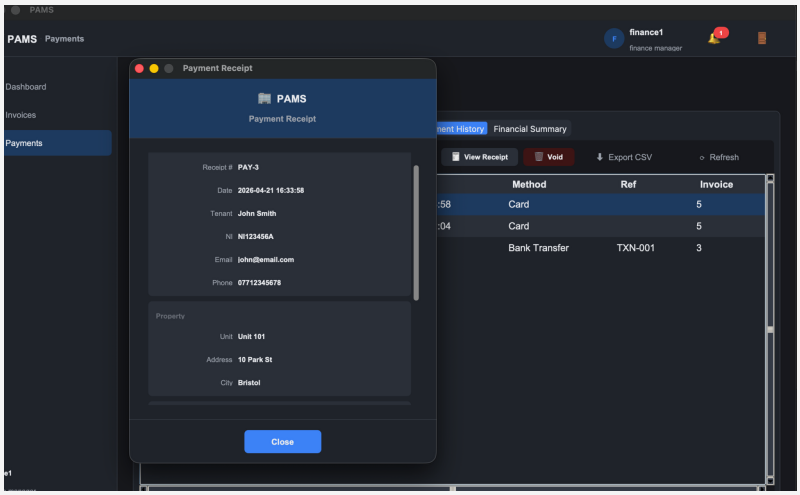
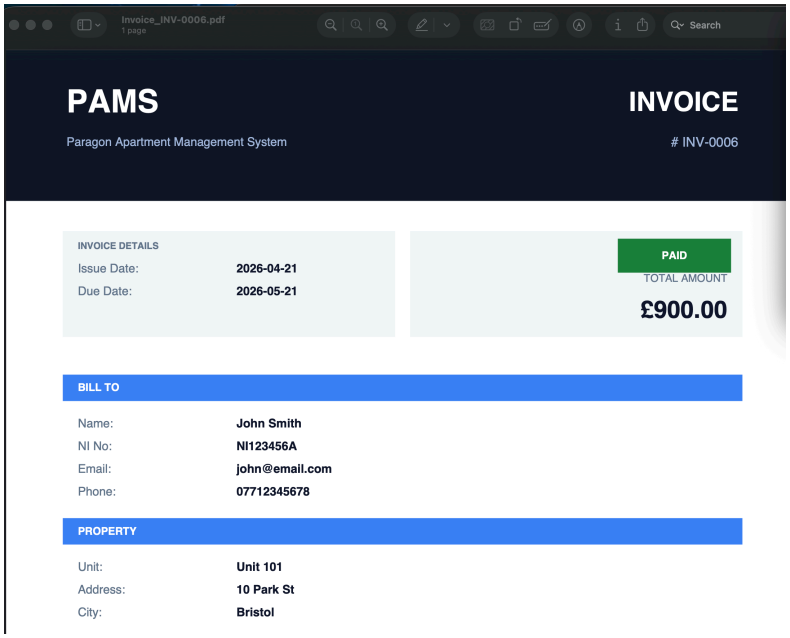
| | | | | |
|---|---|---|---|---|
| INV-08 | Block deletion of a paid invoice | Select a paid invoice, Delete |  | Greys out the delete button and doesn't allow it. |
| INV-09 | Block deletion of an invoice with payments | Select a pending invoice with partial payments, Delete |  | Error toast: "Cannot delete an invoice that has recorded payments." |

## 2.6 Payment Management (PAY)

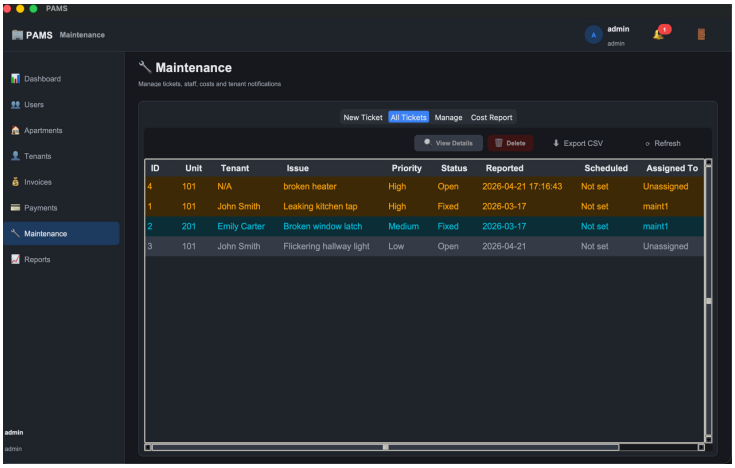
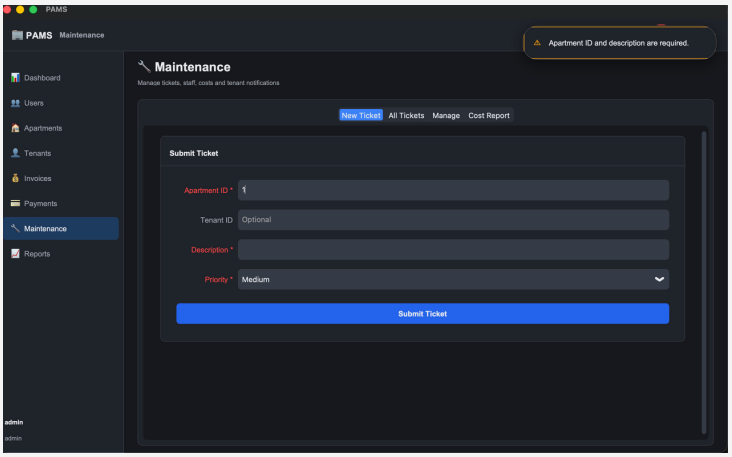
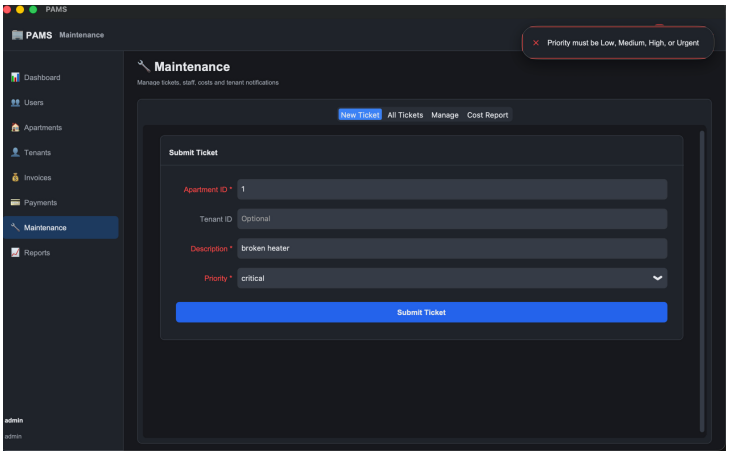
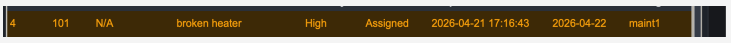
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| PAY-01 | Record a full valid payment | Invoice ID: an existing pending invoice, Amount: full total, Method: `Card`, Reference: `TX001` |  | Success toast. Invoice status flips to `Paid`. Payment appears in Payment History. |

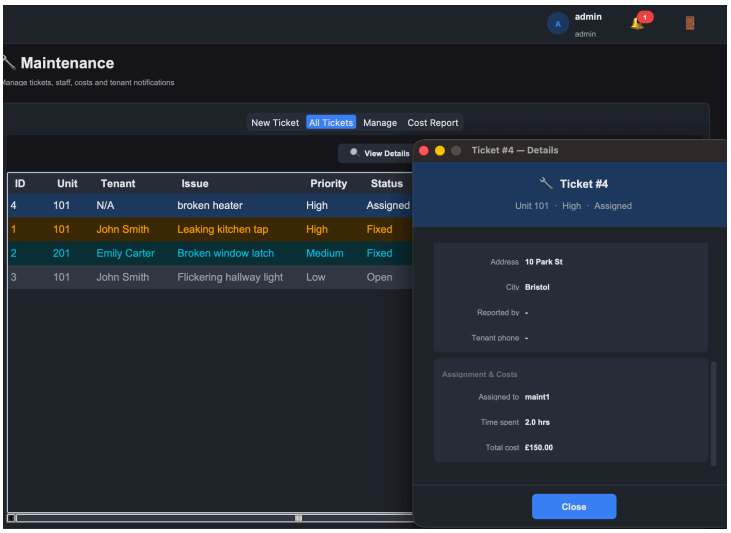
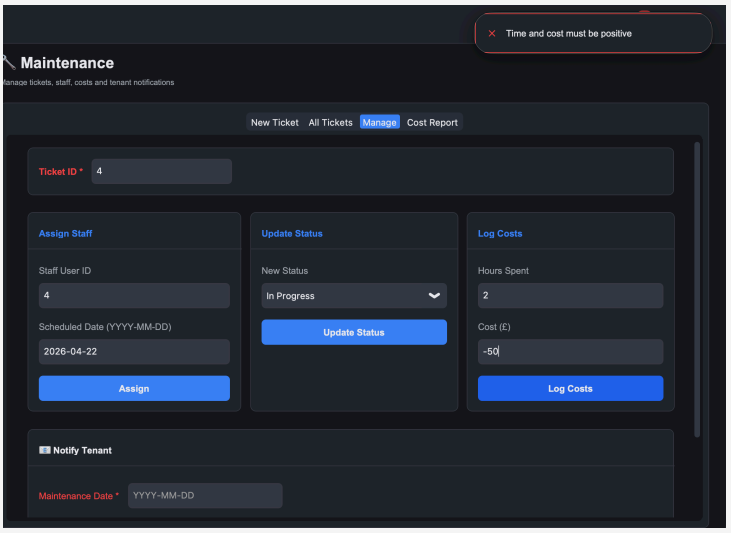
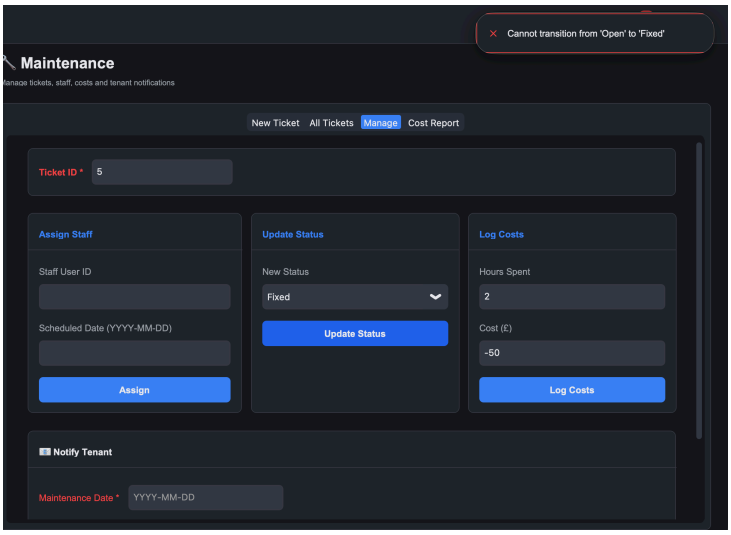
| | | | | |
|---|---|---|---|---|
| PAY-02 | Block payment on an already paid invoice | Select the same invoice and try to pay again |  | Error toast: "Invoice already paid." |
| PAY-03 | Reject a negative payment amount | Amount: -50 |  | Error toast: "Amount must be positive." |
| PAY-04 | Reject zero payment amount | Amount: 0 |  | Error toast: "Amount must be positive." |

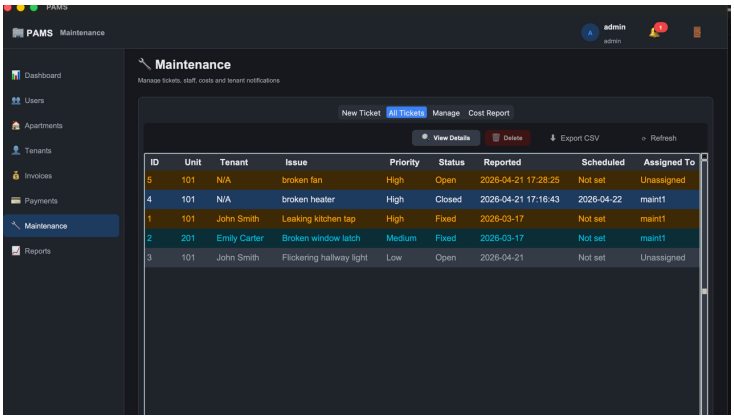
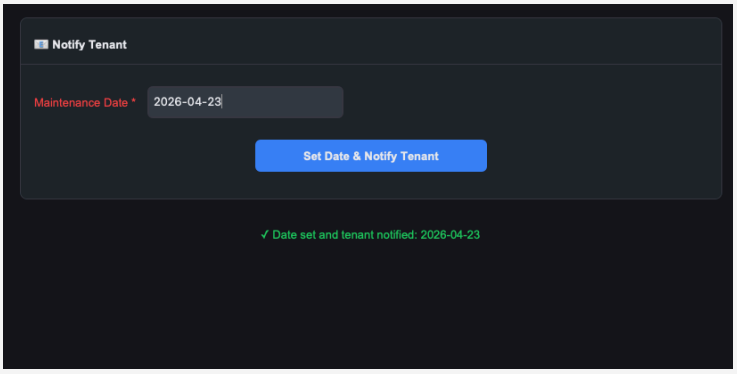
| | | | | |
|---|---|---|---|---|
| PAY-05 | Reject overpayment | Invoice total £900, already paid £500, try to pay £500 |  | Error toast: "Amount exceeds remaining balance of 400.00." |
| PAY-06 | Partial payment keeps invoice Pending | Invoice £900, pay £400 |  | Payment recorded. Invoice status stays `Pending`. |
| PAY-07 | Cumulative payments auto mark invoice Paid | Continue PAY-06, pay remaining £500 |  | Invoice status flips to `Paid` when payments reach the total. |

| | | | | |
|---|---|---|---|---|
| PAY-08 | Receipt popup after payment | Right after PAY-01, click View Receipt |  | Receipt dialog shows tenant name, apartment unit, invoice ID, amount, method, reference and date. |
| PAY-09 | Export receipt as PDF | Click Export PDF on the receipt popup |  | PDF file is saved to the chosen path and opens cleanly. |

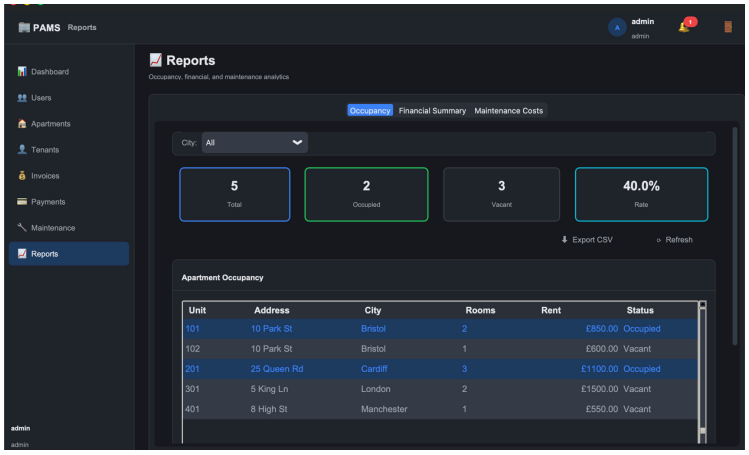
## 2.7 Maintenance Management (MNT)

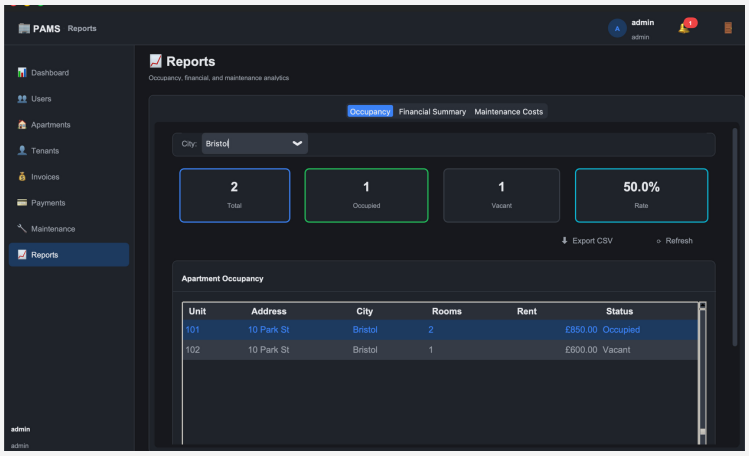
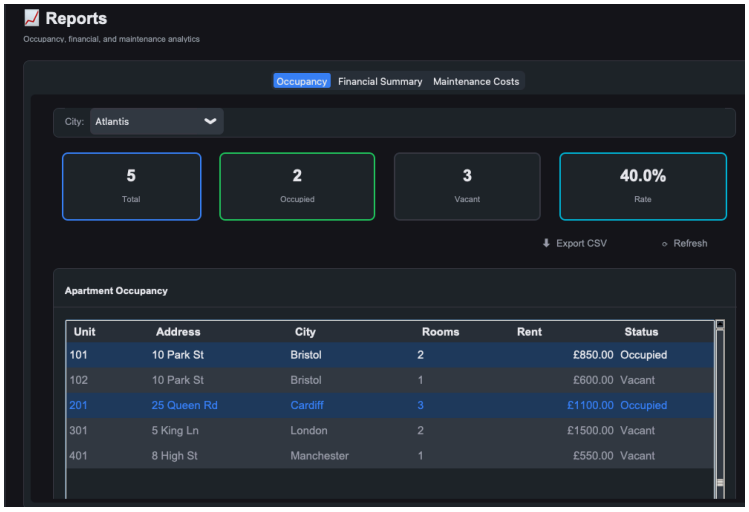
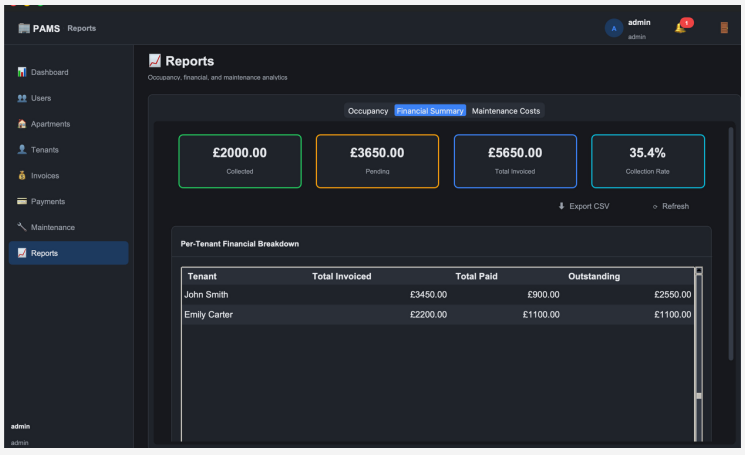
| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| MNT-01 | Create a new ticket | Apartment: `101`, Description: `Broken heater`, Priority: `High` |  | Success toast. Ticket appears in All Tickets with status `Open` and a coloured priority pill. |
| MNT-02 | Reject empty description | Apartment: `101`, Description: (empty) |  | Error toast: "Description is required." |
| MNT-03 | Reject invalid priority | Priority: `Critical` |  | Error toast: "Priority must be Low, Medium, High, or Urgent." |
| MNT-04 | Assign staff to a ticket | Ticket ID: `1`, Staff: `maint1`, Scheduled date: tomorrow |  | Status pill moves from `Open` to `Assigned`. Row shows the assigned staff username. |

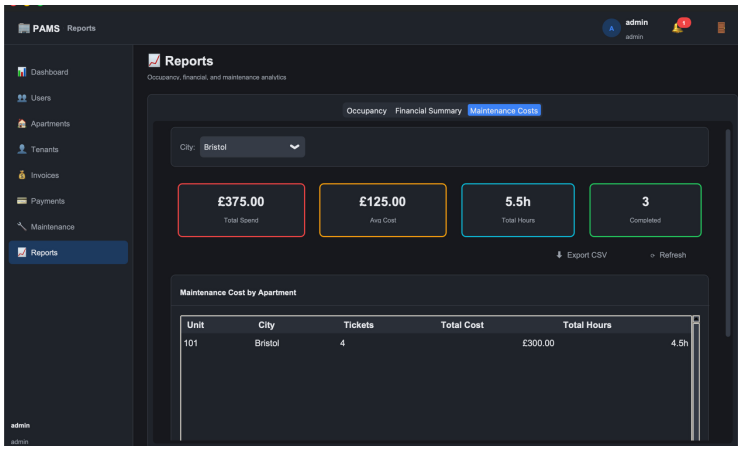
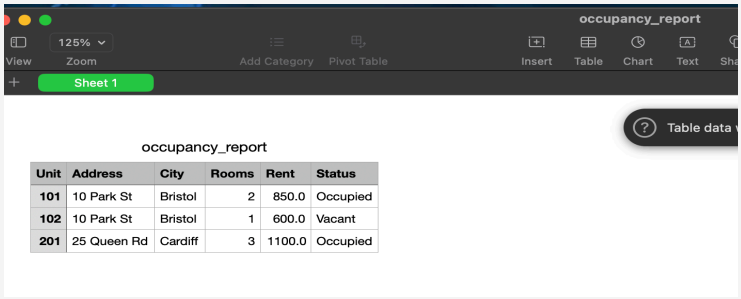
| | | | | |
|---|---|---|---|---|
| MNT-05 | Log material costs | Ticket ID: `1`, Time spent: `2` hours, Cost: `150` |  | Ticket cost increases by £150 and hours by 2. Logging is additive. |
| MNT-06 | Reject negative cost | Ticket ID: `1`, Cost: `-50` |  | Error toast: "Time and cost must be positive." |
| MNT-07 | Block invalid state transition | Ticket in `Open`, attempt direct transition to `Fixed` |  | Error toast: "Cannot transition from 'Open' to 'Fixed'." |
| MNT-08 | Full lifecycle transitions | Open to Assigned to In Progress to Fixed to Closed |  | Every transition succeeds. `date_resolved` is set on `Fixed`. `Closed` is terminal. |

| | | | | |
|---|---|---|---|---|
| MNT-09 | Block deletion of a closed ticket | Select a `Closed` ticket, Delete |  | The delete button is not clickable |
| MNT-10 | Confirmation message | Ticket ID: `1`, Scheduled date: tomorrow, click Notify Tenant |  | Date set and tenant notified: <date> |

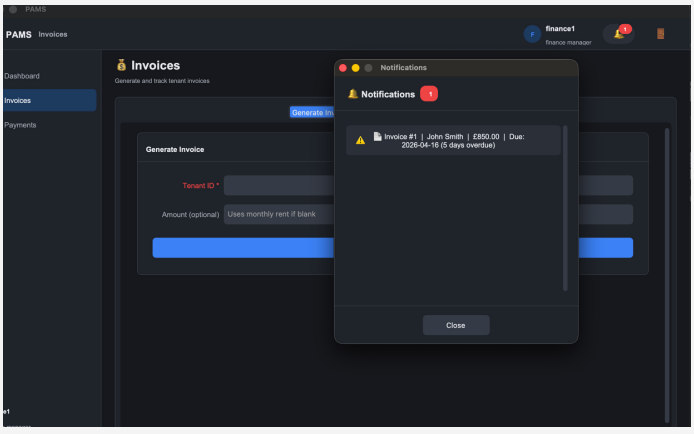
## 2.8 Reports (RPT)

| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| RPT-01 | Occupancy report across all cities | Reports > Occupancy, City: `All` |  | Table lists every apartment. KPI tiles show Total, Occupied, Vacant, Rate %. |

| | | | | |
|---|---|---|---|---|
| RPT-02 | Occupancy report filtered by city | City: `Bristol` |  | Only Bristol apartments listed. KPI tiles recompute. |
| RPT-03 | Empty report on a city with no apartments | type a city with zero apartments |  | Table doesn’t accept it and doesn't filter. |
| RPT-04 | Financial summary report | Reports > Financial Summary |  | KPI tiles show Collected, Pending, Total Invoiced and Collection Rate %. Per tenant table populated. |

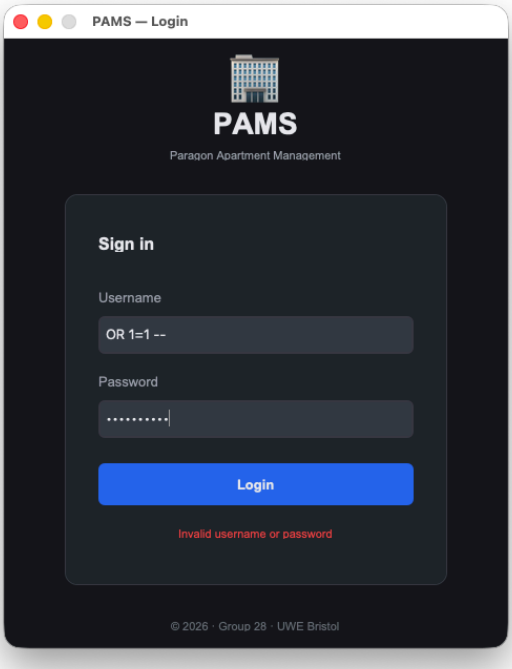
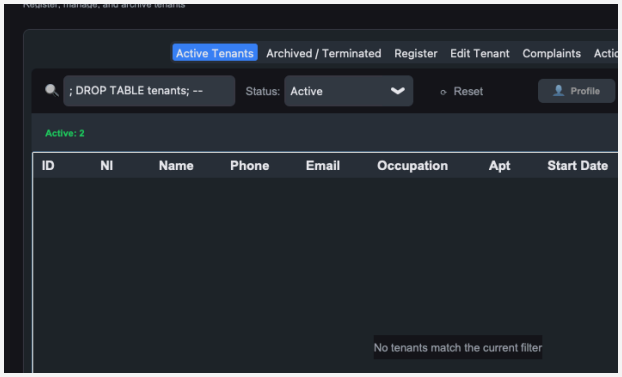
| | | | | |
|---|---|---|---|---|
| RPT-05 | Maintenance cost report by city | Reports > Maintenance Costs, City: `Bristol` |  | Table groups tickets by apartment with total tickets, cost and hours, ordered by cost. |
| RPT-06 | CSV export of occupancy report | Click Export CSV, choose path |  | CSV file is written with correct headers and opens cleanly in a spreadsheet application. |

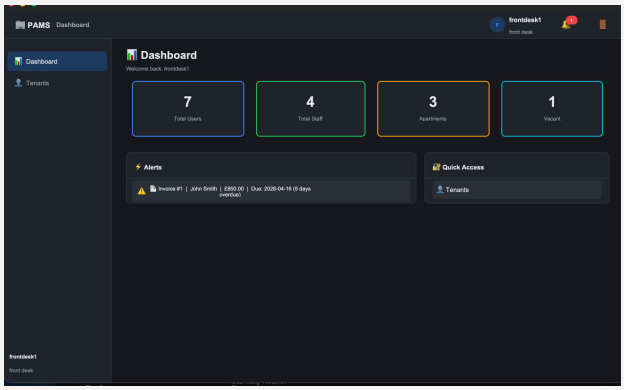
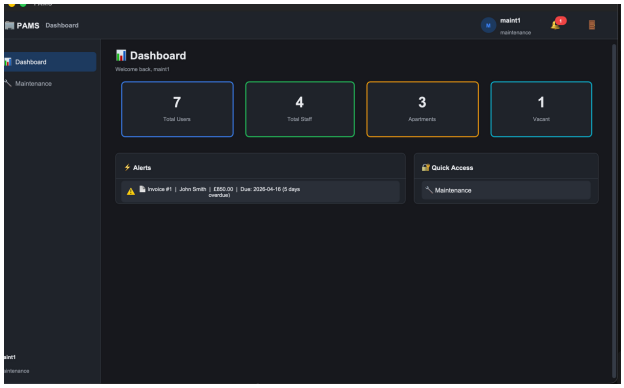
## 2.9 Notifications (NOTIF)

| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| NOTIF-01 | Bell badge appears when there are active notifications | Seed data has at least one overdue invoice and one lease expiring within 30 days |  | Red dot appears on the bell icon. |
| NOTIF-02 | Dropdown lists overdue invoices and expiring leases | Click the bell icon |  | Dropdown lists rows with tenant name, amount or unit, due/end date and days overdue/remaining. |

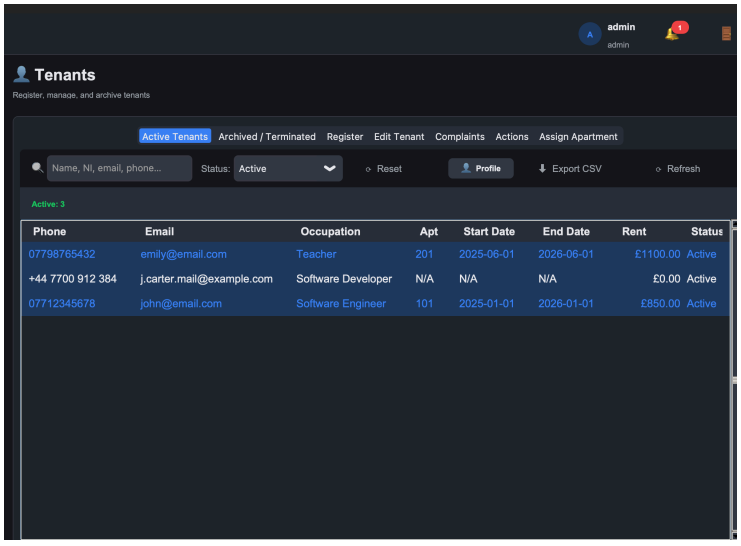
| | | | | |
|---|---|---|---|---|
| NOTIF-03 | Empty state when no notifications | Settle all overdue invoices and ensure no lease expires soon |  | Dropdown shows a caught up message and no red dot on the bell. |

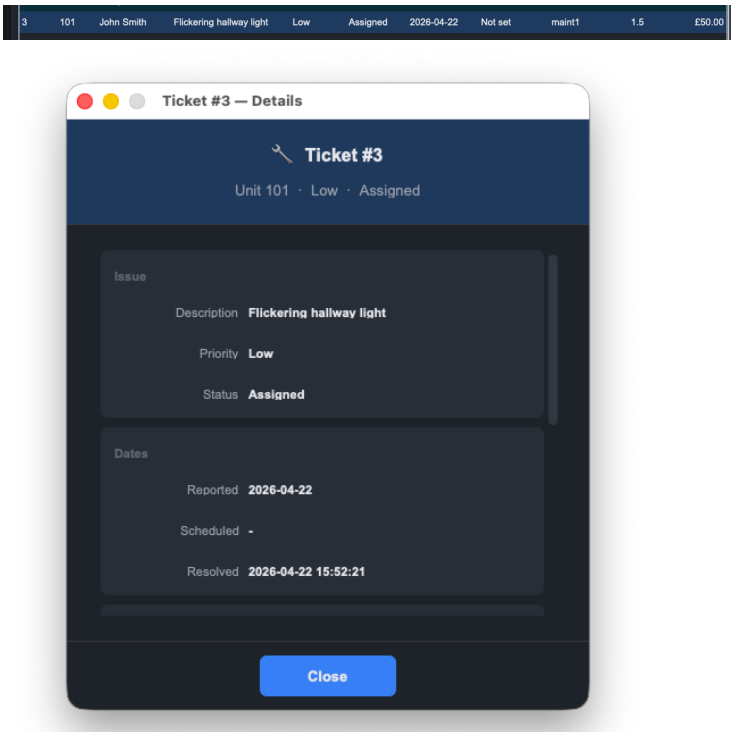
## 2.10 Security (SEC)

| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| SEC-01 | SQL injection attempt on login | Username: `' OR 1=1 --`, Password: `anything` |  | Login fails. The injection string is treated as a literal username by the parameterised query. |
| SEC-02 | SQL injection attempt on tenant search | Search box: `'; DROP TABLE tenants; --` |  | Search returns zero results. Database is intact and the Active Tenants tab continues to load. |
| SEC-03 | Passwords stored as bcrypt hashes | Open `pams.db` in an SQLite viewer and inspect `users.password_hash` |  | All values are 60 character strings starting with `$2b$`. No |

| | | | | |
|---|---|---|---|---|
| | | | | plaintext passwords present. |
| SEC-04 | Front desk staff cannot see the Users tab | Log in as `frontdesk1` and inspect the sidebar |  | The Users tab is not rendered. |
| SEC-05 | Maintenance staff cannot see Finance or Reports | Log in as `maint1` |  | Invoices, Payments and Reports tabs are not rendered in the sidebar. |

## 2.11 Integration Flows (INT)

| Test Case # | Description | Test Dataset / Input | Actual Output | Expected Output |
|---|---|---|---|---|
| INT-01 | Register, assign, invoice, pay, receipt | New tenant, assign to Apartment 101, generate invoice, pay in full |  | Every step succeeds. Apartment becomes `Occupied`, invoice becomes `Paid` |
| INT-02 | Early termination cascade | Follow TEN-12 for a tenant assigned |  | Tenant status `Terminated`, apartment status `Vacant`, penalty equals |

| | | | | |
|---|---|---|---|---|
| | | to an apartment | | rent times 0.05, effective date today plus 30. |
| INT-03 | Full maintenance lifecycle | Create ticket, assign, progress, log cost, fix, close |  | Ticket reaches `Closed` with `date_resolved` set. Cost Report reflects the accumulated cost and time. |

# 3. Automated Testing

## 3.1 Framework

Tests use Python's standard `unittest` framework. All tests live in the `tests/` directory. Every test class inherits from `tests/test_base.py:BaseTest`, which creates a fresh SQLite file at `tests/test_temp.db` in `setUp()` and removes it in `tearDown()`, so tests are deterministic and order independent.

To run the full suite from the project root:

```
python3 -m unittest discover -s tests -v
```

## 3.2 Coverage Summary

| # | Module / Purpose | Test File | Sprint | Tests | Status |
|---|---|---|---|---|---|
| 1 | Authentication | `test_auth_manager.py` | 1 | 21 | Pass |
| 2 | Property / Apartments | `test_property_manager.py` | 1 | 10 | Pass |

| | | | | | |
|---|---|---|---|---|---|
| 3 | Tenant (core) | `test_tenant_manager.py` | 1 | 4 | Pass |
| 4 | Finance (invoice and payment) | `test_finance.py` | 1 | 7 | Pass |
| 5 | Maintenance | `test_maintenance.py` | 1 | 7 | Pass |
| 6 | Reports and Notifications | `test_reports_notifications.py` | 2 | 27 | Pass |
| 7 | Ahmed Sprint 2 (finance analytics) | `test_ahmed_sprint2.py` | 2 | 39 | Pass |
| 8 | Numair Sprint 2 (maintenance analytics) | `test_numair_sprint2.py` | 2 | 40 | Pass |
| 9 | Sprint 3 core | `test_sprint3.py` | 3 | 32 | Pass |
| 10 | Ahmed Sprint 3 (receipts) | `test_ahmed_sprint3.py` | 3 | 8 | Pass |
| 11 | Numair Sprint 3 (edge cases) | `test_numair_sprint3.py` | 3 | 16 | Pass |
| 12 | Coverage gaps | `test_coverage_gaps.py` | 4 | 54 | Pass |
| 13 | CSV export | `test_csv_export.py` | 4 | 4 | Pass |
| 14 | Invoice v2 | `test_invoice_manager_v2.py` | 4 | 10 | Pass |
| 15 | Invoice PDF | `test_invoice_pdf.py` | 4 | 10 | Pass |
| 16 | Invoice search | `test_invoice_search.py` | 4 | 10 | Pass |
| 17 | Maintenance v2 | `test_maintenance_v2.py` | 4 | 12 | Pass |
| 18 | Payment v2 | `test_payment_manager_v2.py` | 4 | 8 | Pass |
| 19 | Property v2 | `test_property_manager_v2.py` | 4 | 7 | Pass |
| 20 | Tenant reactivate | `test_tenant_reactivate.py` | 4 | 4 | Pass |
| | Total | | | **330** | **Pass** |

## 3.3 Run Evidence

Final lines of the runner output:

```
----------------------------------------------------------------------
Ran 330 tests in 86.897s

OK
```

```
test_reactivate_nonexistent_raises (test_tenant_reac
tivate.TestTenantReactivate.test_reactivate_nonexist
ent_raises) ... ok
test_reactivate_terminated_raises (test_tenant_react
ivate.TestTenantReactivate.test_reactivate_terminate
d_raises) ... ok

----------------------------------------------------
----------------------
Ran 330 tests in 86.897s

OK
```

# 4. Conclusion

PAMS has been tested thoroughly at two levels. Manual testing covers every component the case study lists (account and user management, tenant, apartment, invoice and payment, maintenance, reporting) plus security and end-to-end integration, with screenshot evidence for every case. Automated testing (330 tests across 20 files, all passing) locks in business rules, validation and edge cases at the manager layer.

Every case listed in Section 2 passed when executed against a freshly seeded database. Every automated test passes on each run. No known defects remain open at the time of submission.

*End of document.*